

Computer Architecture Final Project Report

Team ID: 1
B11901162 高爾謙
B11901164 陳秉緯

1. Design Specification

Phase 1: Dynamic Programming

- Constructs cost table $M[i][j]$ representing the minimum cost to multiply matrices from position i to j (count $\times$ count $\times$ 4 bytes).
- Builds split table $S[i][j]$ storing the optimal split point k for the subsequence from i to j (count $\times$ count $\times$ 4 bytes).
- Uses bottom-up approach with increasing chain lengths ($l = 2$ to count).

Phase 2: Matrix Multiplication

- Builds result table containing intermediate and final matrix products (count $\times$ count $\times$ 4 bytes).
- Follows optimal parenthesization determined by the S table.
- Performs actual matrix multiplications using the computed split points.
- Total memory footprint: $3 \times \text{count}^2 \times 4$ bytes plus individual matrix storage.

Input/Output Specification

input	matrices	a0	Array of matrix pointers
	rows	a1	Array of row dimensions for each matrix
	cols	a2	Array of column dimensions for each matrix
	count	a3	Number of matrices in the chain
output	a0	Pointer to the final result matrix	

2. Special Techniques and Optimizations

2.1 Memory Management Optimization

- Single Allocation Strategy:

```
mul t0, s3, s3    # count * count
slli t1, t0, 3    # * 8 (for both M and S tables)
mv a0, t1
call malloc
mv s4, a0         # s4 = M table
add s5, s4, t0    # s5 = S table (offset from M)
```

The M and S tables are allocated in a single contiguous memory block, reducing malloc overhead and improving cache locality. The S table is positioned immediately after the M table in memory.

2.2 Enhanced Dynamic Programming Optimizations

- Row Base Address Caching:

```
# Initialize M[i][j] to infinity
li t0, 0x7FFFFFFF
mul t1, s7, s3    # i * count
add t5, t1, s8    # i * count + j
slli t5, t5, 2    # * 4
add t6, s5, t5    # &S[i][j] stored in t6
add t5, s4, t5    # &M[i][j] stored in t5
sw t0, 0(t5)      # M[i][j] = infinity

# Try all split points k = i to j-1
addi s11, s7, -1  # k-1 = i-1

# &M[i][k] stored in a5
add t0, t1, s11   # i * count + k
slli t0, t0, 2    # * 4
add a5, s4, t0    # &M[i][k]
```

Computer Architecture Final Project Report

Team ID: 1
B11901162 高爾謙
B11901164 陳秉緯

```
# &M[k+1][j] stored in a6
mv s11, s7      # k = i

mul t0, s11, s3  # (k+1) * count
add t0, t0, s8   # (k+1) * count + j
slli t0, t0, 2   # * 4
add a6, s4, t0   # &M[k+1][j]
```

With pre-computing row base addresses for the $M[i][j]$, $S[i][j]$, $M[i][k]$, $M[k+1][j]$ address in DP I-loop, redundant address calculations were eliminated and memory access efficiency was improved. Furthermore, computing $M[i][j]$, $S[i][j]$, $M[i][k]$ at same time can minimize the calculations.

- **Pre-computed values in outer loop**

(a)

Pre-calculated Stride Values:

```
# Main DP loop
li s6, 2      # l = 2 (chain length)
slli a7, s3, 2 # 4*count, using in (M[k+1][j])
```

Countdown Loop Optimization:

```
# Get M[k+1][j]
add a6, a6, a7 # &M[k][j]+4*count= &M[k+1][j]
lw t2, 0(a6)   # t2 = M[k+1][j]
```

Computing the value $4*n$ before DP loop can minimize the $M[k+1][j]$ address calculations in the innermost k-loop of DP.

```
# Calculate j = i + l - 1
addi s8, s6, -1 # j0=i+1-1=1-1 end i_loop with i++ and j++
# for_i boundary
sub a4, s3, s6 # count - 1
addi a4, a4, 1 # count - 1 + 1
```

(b)

Pre-computing the value j and the end point of the i -loop in the outer i -loop can reduce the calculations in i -loop.

2.3 Cache-Aware Memory Access Patterns

- **Dimension Value Caching:**

```
slli t1, s7, 2 # i * 4
add t1, s1, t1 # &rows[i]
lw s9, 0(t1)   # s9 = rows[i] (cache for k loop)

slli t1, s8, 2 # j * 4
add t1, s2, t1 # &cols[j]
lw s10, 0(t1)  # s10 = cols[j] (cache for k loop)
```

Frequently accessed dimension values are loaded once and cached in saved registers ($s9$, $s10$) throughout the inner k -loop, reducing memory accesses in the DP phase.

Computer Architecture

Final Project Report

Team ID: 1
B11901162 高爾謙
B11901164 陳秉緯

2.4 Optimized Matrix Multiplication Implementation

Pre-calculated Stride Values:

```
# Pre-calculate constants to eliminate inner loop calculations
slli t5, s2, 2 # cols_right * 4 (B row stride)
slli t3, s1, 2 # cols_left * 4 (A row stride)
```

Countdown Loop Optimization:

```
mv t2, s1          # k = cols_left
mult_k_loop:
    lw a4, 0(a2)     # A[i][k]
    lw a5, 0(a3)     # B[k][j]
    addi a2, a2, 4    # advance A pointer
    add a3, a3, t5    # advance B pointer
    mul a6, a4, a5    # A[i][k] * B[k][j]
    add a1, a1, a6    # sum += product
    addi t2, t2, -1   # k--
    bnez t2, mult_k_loop
```

The matrix multiplication uses countdown loops and pointer arithmetic to minimize address calculations. Stride values are pre-computed outside the loops, and the innermost loop uses direct pointer advancement instead of index-based addressing.

2.5 Register Allocation Strategy

The implementation employs an optimized register allocation strategy:

① Saved Registers (s0-s11):

- s0-s3: Input parameters (persistent across function calls)
- s4-s5: DP table base addresses
- s6-s8: Loop control variables
- s9-s10: Cached dimension values
- s11: Inner loop iterator

② Temporary Registers (t0-t6):

- Address calculations and intermediate values
- Pre-computed stride values
- Short-lived computations within basic blocks

③ Function Call Registers (a0-a7):

- a7: Pre-computed stride values in DP. Result matrix pointer in multiplication function.
- Optimized parameter passing to reduce register pressure

3. Performance Analysis

3.1 Performance Results

Execution Time	2395864 ns
Score	69480056.0
Target Achievement	Successfully below the strong baseline: 70,000,000 thresholds

Computer Architecture

Final Project Report

Team ID: 1
B11901162 高爾謙
B11901164 陳秉緯

3.2 Cache Performance Characteristics

The implementation is optimized for the final cache configuration that achieved the target performance:

L1 Instruction Cache	4kB, 64-way associative
L1 Data Cache	4kB, 64-way associative
L2 Cache	1kB, 16-way associative

4. Team Division of Work

- 高爾謙：Dynamic programming
- 陳秉緯：Matrix multiplication

5. Reflections

- 高爾謙：

This final project was a big challenge to me. I encountered many bugs immediately when writing the DP code, which took me a long time to overcome. After the codes were completed, optimization was another tough problem. We rearranged the calculation order, reduced the calculations in the loop, and found better combinations of Cache size again and again. Many thanks for my strong teammate for completing the most difficult and contributing part of the program. Eventually, we made the result below the strong baseline.

- 陳秉緯：

I am very grateful to have a responsible teammate, who did not give up like teammates in other classes. I feel that writing assembly code is more difficult to debug and design logic than simple C/C++. When I first started writing, I encountered a lot of strange bugs, so I switched to using simple matrix multiplication for testing. I especially thanked my teammate for later solving the bugs and successfully implementing DP, which allowed me to simply multiply the results according to the S table. In the end, we kept optimizing the code together to pass the strong baseline, and I am very happy that we finally succeeded in lowering the score.